Building Firebase Apps in Google Antigravity IDE

Fulllion Creative Works

A working guide to developing Firebase apps locally with the Google Antigravity IDE — whether you're migrating an existing Firebase Studio project or starting from scratch. Covers Cloud Firestore, the Local Emulator Suite, security rules unit tests, and automated deployments.

> **If you're coming from Firebase Studio:** new workspace creation was disabled on June 22, 2026 and the editor sunsets on March 22, 2027. Apps already deployed to Firebase keep running, but the editor goes away — so everything below is about making your local setup self-sufficient. Migration steps are in Section 3; new projects start at Section 4.

Contents

1. Local Runtime Environment Setup
2. Install Antigravity and Connect It to Firebase
3. Path 1 — Migrate an Existing Firebase Studio Project
4. Path 2 — Start a New Project in Antigravity
5. Working With the Agent: Prompts That Do Real Work
6. Choose Your Deployment Path
7. Production Firestore Security Rules
8. Local Emulator Seeding
9. Firestore Rules Unit Tests
10. Continuous Deployment
11. Next Steps

---

1. Local Runtime Environment Setup

Antigravity and the Firebase CLI both require **Node.js 20 or higher**, and the migration workflow requires **Firebase CLI 15.10.0 or higher**.

Windows

1. Download the Node.js LTS installer from nodejs.org.
2. Run the installer and accept the defaults. Leaving **"Automatically install the necessary tools"** checked pulls in Chocolatey and native build dependencies, which some npm packages need.
3. Open PowerShell and verify:

```
node -v
npm -v
```

If `node -v` works but `npm -v` doesn't

This happens often enough after the MSI installer that it's worth knowing the two causes. Read the error message — it tells you which one you have.

Error mentions `npm.ps1` or "running scripts is disabled" — PowerShell's execution policy is blocking the npm launcher script. Node and npm are both installed and on PATH; PowerShell just refuses to run `.ps1` files. Fix it for your user account:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

Then rerun `npm -v`. If you'd rather not change the policy, `npm.cmd -v` works immediately and always.

Error says "npm is not recognized" — the npm folder isn't on PATH. First try a fresh terminal: the installer updates PATH, but windows opened before the install don't see it. If a new window still fails, add the paths for the current session to confirm that's the problem:

```
$env:Path += ";$env:ProgramFiles\nodejs;$env:AppData\npm"
npm -v
```

If that works, make it permanent:

1. Press `Win + R`, type `sysdm.cpl`, press Enter.
2. **Advanced** tab → **Environment Variables…**
3. Under **User variables**, select **Path** → **Edit…** → **New** → add `%AppData%\npm`.
4. Under **System variables**, select **Path** → **Edit…** and confirm `C:\Program Files\nodejs\` is present. Add it if not.
5. **OK** through every dialog, then open a **new** PowerShell window.

Verify with `where.exe npm` — it should print a path under `C:\Program Files\nodejs\`.

Nothing works — run the Node.js MSI again and choose **Repair**, then reboot. As a last resort, nvm-windows manages Node and npm together and sidesteps PATH issues entirely, but uninstall the MSI version first or the two will fight.

macOS

1. Install Homebrew if you don't have it:

   ```
   /bin/bash -c "$(curl -fsSL
   https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
   ```

2. Install and link Node.js 20:

   ```
   brew install node@20
   brew link node@20
   ```

3. Verify:

```
node -v
npm -v
```

Linux (Ubuntu / Debian)

1. Register the NodeSource repository:

```
sudo apt-get update && sudo apt-get install -y curl gnupg
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://deb.nodesource.com/gpgkey/nodesource-repo.gpg.key \
  | sudo gpg --dearmor -o /etc/apt/keyrings/nodesource.gpg
echo "deb [signed-by=/etc/apt/keyrings/nodesource.gpg] https://deb.nodesource.com/node_20.x
nodistro main" \
  | sudo tee /etc/apt/sources.list.d/nodesource.list
```

2. Install Node.js and build tools:

```
sudo apt-get update
sudo apt-get install -y nodejs build-essential
```

3. Verify:

```
node -v
npm -v
```

Firebase CLI

Install globally, then confirm the version meets the 15.10.0 minimum:

```
npm install -g firebase-tools
firebase --version
```

The install prints a wall of warnings. As long as you see `added N packages`, it succeeded — the rest can be ignored:

- **`npm warn deprecated …`** — refers to packages inside firebase-tools' own dependency tree. Those are Google's to update; nothing you can do from your side.
- **`npm warn install-scripts …`** — newer npm versions skip post-install scripts from indirect dependencies by default. The two it names (`protobufjs`, `re2`) are optional for the Firebase CLI. If you later hit an error mentioning `re2`, rerun with `npm install -g --allow-scripts=protobufjs,re2 firebase-tools`.
- **`New major version of npm available`** — optional. Nothing in this guide requires it.

If you hit permission errors on macOS or Linux, prefix with `sudo` (or fix your npm prefix — the cleaner fix long-term).

Authenticate:

```
firebase login
```

The CLI asks two questions before opening a browser:

1. **Enable Gemini in Firebase features?** — say **Yes**. This is what lets the Firebase MCP server (Section 2) use Gemini-backed tools inside Antigravity. Saying no here means some agent features won't work.
2. **Allow Firebase to collect CLI and Emulator Suite usage and error reporting information?** — your call. It's anonymous telemetry; **No** is fine and changes nothing functionally.

It then prints a long `accounts.google.com` URL and opens it in your default browser. Sign in with the Google account that owns (or will own) your Firebase project and approve the requested permissions. The browser redirects to `localhost:9005`, the terminal prints `Success! Logged in as <your-email>`, and you're done.

If the browser doesn't open, copy the URL from the terminal and paste it in manually. If you're on a machine without a browser (SSH, WSL without a display), use `firebase login --no-localhost` instead — it gives you a code to paste back into the terminal.

To change either answer later: `firebase logout`, then `firebase login` again.

Java (for the emulators)

The Firestore emulator runs on the JVM. Install a current JDK if `java -version` fails — the emulator will tell you which minimum version it needs when it starts. On macOS, `brew install openjdk` works; on Ubuntu, `sudo apt-get install -y default-jdk`.

---

2. Install Antigravity and Connect It to Firebase

Do this once, regardless of whether you're migrating or starting fresh.

Install the IDE

1. Download the installer for your OS from antigravity.google/download.
2. Run it and open Antigravity. Sign in with your Google account when prompted.
3. During onboarding you'll be offered **Build with Google** integrations — tick **Firebase** if you see it. If you skip it, you can enable it later (next step).

Enable the Firebase Bundle (agent skills)

Settings → **Customizations** → **Build with Google Plugins** → enable **Firebase**.

This installs a set of agent *skills*: version-aware instructions that teach the agent how to initialize Firebase services, write security rules, and deploy to App Hosting or Hosting using current best practices. Without this, the agent falls back on whatever it remembers from training, which can be stale.

Install the Firebase MCP server (agent tools)

In the **Agent** pane, open the three-dot (`more_horiz`) menu → **MCP Servers** → find **Firebase** → **Install**.

Skills tell the agent *how*; the MCP server gives it the *hands*. With it installed, the agent can list your Firebase projects, read your web app config, query and write Firestore data, manage Auth users, and run deploys — all through the Firebase CLI credentials you set up with `firebase login` in Section 1.

You can confirm it's working by asking the agent: "*List my Firebase projects.*"

How the agent pane works

A few things worth knowing before you start prompting:

- **Model picker** — bottom of the Agent pane. Gemini Flash is cheaper and faster; use it for mechanical work like file conversion. Use the larger models for design decisions and debugging.
- **Plans and walkthroughs** — for non-trivial tasks the agent produces an implementation plan you review before it touches files. Read them. This is where you catch it doing something you didn't intend.
- **Permissions** — the agent asks before running commands like `firebase deploy`. Say no if a plan looks wrong; it will revise.
- `@workflows <name>` — runs a saved workflow file. `@fbs-to-agy-export` (used in Section 3) is one of these.
- **What it can't see** — the agent works through the CLI and your local files. It cannot open the Google Cloud Console, so when a cloud build fails it will hand you a log link and ask you to paste the error back. It also doesn't automatically know about resources that already exist in your project (like an App Hosting backend created by Studio) unless it goes looking — so mention them in your prompt.
- **Long-running commands** — deploys and dev servers keep running after the agent says it's "monitoring in the background." Check the terminal panel for the real status rather than waiting on the chat.

---

3. Path 1 — Migrate an Existing Firebase Studio Project

Skip to Section 4 if you're starting a new codebase.

Export from Firebase Studio

1. Open your workspace in Firebase Studio and click the **Move now** button at the top.
2. Follow the export method that appears:
   - If you see a **Zip and Download** button, click it.
   - Otherwise open the command palette (`Cmd+Shift+P` on Mac, `Ctrl+Shift+P` elsewhere) and run **Firebase Studio: Zip & Download**.
3. Extract the archive to a local folder.

> If no download window appears, check your browser's address bar for a blocked pop-up.

You don't need `node_modules/` or `.next/` in the download. If the zip is huge, or you're copying the workspace out some other way (a manual download of individual folders, a `cp` from the Studio terminal), skip both. They're generated folders, not source: `node_modules/` is rebuilt from `package.json` by `npm install`, and `.next/` is rebuilt by `npm run build` (or on the fly by `npm run dev`). Together they're typically hundreds of megabytes and tens of thousands of files, and a `node_modules/` built on Studio's Linux image can contain platform-specific binaries that won't work on Windows or macOS anyway. Source, `package.json`, `package-lock.json`, and your config files are all that has to make the trip — everything else comes back with `npm install` and a build once you're local.

Convert the project — Option A, agent-driven

1. Open the extracted folder in Antigravity (**File** → **Open Folder...**).
2. In the Agent pane, select the **Gemini Flash** model.
3. Enter:

   ```
   @fbs-to-agy-export
   ```

   The agent reads the Studio workspace config (including the Nix environment definition), rewrites it into a standard local project layout, and installs anything missing. It will ask for help along the way — follow its guidance. If it errors, tell it to try again.

   When it finishes it prints a report. A successful run looks like this — audit rows for Node, Firebase CLI, login state, the linked project, a clean `tsc --noEmit`, and a dev server that answered HTTP 200 — plus a list of files it wrote:

   - `.firebaserc` — project alias pointing at your Firebase project ID.
   - `firebase.json` — links `firestore.rules` and adds emulator mappings for Auth (9099) and Firestore (8080).
   - `.vscode/launch.json` — `Next.js: dev` and `Next.js: debug` configurations for the Run & Debug panel.

   If any audit row says anything other than passed/active/linked, fix that before moving on; everything downstream assumes those are true.

   If `launch.json` shows a schema error: the `port` property is only valid on *attach* configurations, not *launch* ones. Some exports write `"port": 9229` into the launch configs. Delete that line (or ask the agent to) and the error goes away.

Convert the project — Option B, manual (no AI tokens)

```
npx firebase-tools@latest studio:export <path-to-folder-or-zip>
```

Currently optimized for Next.js, Flutter, and Angular workspaces. Other stacks may need hand-fixing afterward.

Reconnect to your existing Firebase project

Your production data and deployed app are untouched by the export — but the local checkout needs to know which project it belongs to.

1. Check whether `.firebaserc` exists in the project root and names the right project. If not:

   ```
   firebase use --add
   ```

   and pick your project from the list.
2. Ask the agent to verify the wiring:

   > *"Link this app to my existing Firebase project and verify the Firestore configuration. Don't modify any data."*

   It will check `.firebaserc`, `firebase.json`, and your web SDK config against what the MCP server sees in the cloud, and flag mismatches.

Install the project's dependencies

The Studio export doesn't include `node_modules/` or `.next/` — and if you skipped them deliberately per the note above, good — so nothing runs until you install dependencies. (A git clone is the same: both folders are always gitignored.) The `@fbs-to-agy-export` workflow usually does this for you; if you're going the manual route, or `npm run dev` fails with `next: command not found` / `Cannot find module`, do it yourself from the project root:

```
npm install
```

This reads `package.json`, resolves every dependency, downloads them into `node_modules/`, and writes (or updates) `package-lock.json`. Expect it to take a minute or two the first time and to print the same kind of deprecation warnings as the Firebase CLI install did — `added N packages` at the end is what counts. A few things to know:

- **`npm ci` vs `npm install`** — if a `package-lock.json` is already present, `npm ci` installs *exactly* what the lockfile says and fails loudly if `package.json` disagrees. It's what CI uses (Section 10) and what you want when reproducing a known-good state. `npm install` is the everyday command and is what updates the lockfile when you add or change dependencies.
- **Check which package manager the project uses first.** A `pnpm-lock.yaml` means `pnpm install`; `yarn.lock` means `yarn`; `bun.lockb` means `bun install`. Mixing managers produces a second lockfile and subtle version drift. Studio-generated projects use npm.
- **Commit the lockfile.** `package-lock.json` belongs in git; `node_modules/` never does. The lockfile is what makes the cloud build in App Hosting install the same versions you tested locally.
- **When to rerun it** — after pulling changes that touched `package.json`, after switching branches that did, or any time a module-not-found error appears out of nowhere. If things get strange, the nuclear option is safe: delete `node_modules/` and `package-lock.json`, then `npm install` again.

- **Windows note** — on a secondary drive `npm install` can be noticeably slow; that's the same slow-filesystem effect Next.js warns about, not a problem with the install.

Preview locally

Open **Run and Debug** in the left sidebar and click play (F5), or run `npm run dev` in the terminal. Studio-generated Next.js apps usually serve on `http://localhost:9002`; after the export the port is often reset to Next.js's default of `3000`. Read the terminal — it prints the URL. Confirm the app loads and can read from your live Firestore before going further.

Two things about that terminal that confuse people the first time:

- **It looks frozen. It isn't.** `next dev` is a persistent server. Once you see `✓ Ready in 359ms` and a `Local:` URL, it's up and waiting for requests. It only prints more when you open the URL in a browser, at which point routes compile on demand and you'll see lines like `GET / 200`. Close it with `Ctrl+C` when you're done.
- `⚠ Slow filesystem detected` is an informational note from Next.js, common when the project lives on a secondary or external drive. It doesn't stop anything.

Lock down `.gitignore` before your first commit

The export doesn't necessarily leave you with a git repository, so this is the moment to make sure nothing sensitive can be committed later. Ask the agent — "*make sure `.gitignore` blocks env files, service account keys, Firebase credentials, and debug logs*" — or add these yourself:

```
# secrets and local environment
.env*
.genkit/*

# credentials and service account keys
*serviceAccount*.json
*credentials*.json
*-adminsdk-*.json
google-services.json
GoogleService-Info.plist
*.key
*.p8
*.p12
*.pfx

# firebase debug logs
firebase-debug.log*
firestore-debug.log*
ui-debug.log*

# scratch and local tools
tmp/
.claude/
```

The agent may also propose ignoring `firestore.rules` and `.firebaserc`. Think twice before accepting. Neither contains a secret — `.firebaserc` is just your project ID, and rules are access policy, not credentials — and **the CI workflows in Section 10 need `firestore.rules` in the repository** to test and deploy it. If your repo is private, commit both. Ignore them only if the repo is public and you'd rather not advertise your data model; then keep a copy somewhere safe, because losing the rules file means reconstructing it from the console.

Note that "confirm no secrets are in source" checks are meaningless until `git init` has actually run — the agent can only check what git tracks. If you haven't initialized a repo yet, do it now (`git init`), then check `git status` and make sure `.env.local` and friends aren't listed.

Republish

Once the local preview works:

> "*Publish my app*"

When the agent asks to run `firebase deploy`, choose **Yes**. It should detect whether you previously deployed to App Hosting or Hosting and update your existing live URL. If this is the first App Hosting deploy from this machine, it walks you through creating the backend.

Tell it about your existing backend up front. In practice the agent doesn't always find the backend Studio created — it may answer the CLI's prompts itself and provision a brand-new one in a different region, leaving you with two backends and a second URL. Before you publish, find out what you have:

```
firebase apphosting:backends:list
```

Studio-created backends are usually named `studio` in `us-central1`. Then be explicit:

> "*Publish my app to my existing App Hosting backend `studio` in `us-central1`. Don't create a new backend.*"

If it already created a stray one, delete it from the console (**App Hosting** → **backend** → **Settings**) or with `firebase apphosting:backends:delete <id>` so you aren't billed for two.

If the cloud build fails after a local build succeeded. The failure message includes a Cloud Build log link; the agent can't open the Cloud Console, so click it yourself and paste the last error into the chat. The most common cause is missing build-time environment variables. Your `firebase.ts` reads `NEXT_PUBLIC_FIREBASE_*` from `process.env`, and Next.js runs that code while prerendering pages during `npm run build` — so the values have to be present in the cloud builder, not just in your local `.env.local`, which is gitignored and never uploaded. The fix is to declare them in `apphosting.yaml`:

```
env:
  - variable: NEXT_PUBLIC_FIREBASE_API_KEY
    value: your-web-api-key
  - variable: NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN
    value: your-project.firebaseapp.com
  - variable: NEXT_PUBLIC_FIREBASE_PROJECT_ID
    value: your-project
  - variable: NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET
    value: your-project.appspot.com
  - variable: NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID
    value: "1234567890"
  - variable: NEXT_PUBLIC_FIREBASE_APP_ID
    value: 1:1234567890:web:abc123
```

Plain `value:` is fine for these — they're the public web config that ships to every browser anyway. Anything actually secret (`GEMINI_API_KEY`, server-side keys) must use `secret:` and Cloud Secret Manager instead; see Section 6, Path A. The second most common cause is a Node runtime mismatch between an older backend and a newer Next.js; the build log will say so.

Then redeploy — `npx firebase-tools deploy` from the terminal, or ask the agent. A successful App Hosting deploy also pushes `firestore.rules`, and the live URL follows the pattern `https://<backend>--<project-id>.<region>.hosted.app`.

What about the files still in Firebase Studio?

After deploying from your machine you'll notice the Studio editor still shows the old files. That's expected: `firebase deploy` uploads directly from your local folder, and nothing syncs back to Studio. There is no need to update it — Studio is being sunset (see the top of this guide) and your local folder is now the source of truth. Don't edit in both places.

What you should do instead is get the local folder under version control so it exists somewhere other than one hard drive:

```
git init
git add .
git status        # confirm no .env* or key files are staged
git commit -m "Migrate from Firebase Studio to Antigravity"
```

Then create a GitHub repo and push. Once it's on GitHub you can also connect the App Hosting backend to the repo for automatic rollouts (Section 10, Path A).

Publishing from the command line. Two options, depending on whether you have the GitHub CLI installed.

With `gh` (install from cli.github.com, then `gh auth login` once):

```
gh repo create my-app --private --source=. --remote=origin --push
```

That creates the private repo, adds it as `origin`, and pushes your current branch in one step. Add `--description "..."` if you want one.

Without `gh`: create an empty **private** repository at github.com/new — leave "Add a README" unchecked so there's nothing to merge — then point your local repo at it and push:

```
git branch -M main
git remote add origin https://github.com/<your-username>/my-app.git
git push -u origin main
```

Git will prompt for credentials on the first push over HTTPS; use a personal access token, not your account password (GitHub rejects passwords for git operations). After that first `-u` push, subsequent updates are just:

```
git add .
git commit -m "Describe the change"
git push
```

Run `git status` before every `git add .` until you trust your `.gitignore` — it's the cheapest secrets audit there is.

You can also hand all of this to the agent ("*commit these changes and push to GitHub*"), but it pauses for permission before each `git push`, so expect to approve it.

If you use GitHub Desktop, let the agent run `git init` and stage the files, then do the rest in the app — it's the easiest way to get a *private* repo without touching the `gh` CLI:

1. **File → Add local repository...** (`Ctrl+O`), point it at the project folder.
2. Your files appear as an initial commit. Review the list — this is your last check that no `.env*` or key files slipped through — then **Commit to main**.
3. Click **Publish repository**, tick **Keep this code private**, add a description, and publish.

Before that first commit, also ignore the things that only mattered in Studio or to the agent and don't contribute to the build: the `.idx/` folder (Studio's workspace config, dead weight now), `CLAUDE.md` or other AI-assistant notes, and `.claude/`. Ask the agent to add them to `.gitignore` *and* unstage them — adding a pattern to `.gitignore` doesn't remove files that are already staged.

Coming back to this project later

Everything the agent needs to redeploy lives in the project files, not in the chat history, so switching to another project and returning weeks later is fine. What matters:

- **Open the right folder.** The agent's working directory is whatever's open in Antigravity. With the project folder open it reads `.firebaserc` (which Firebase project), `firebase.json` (which App Hosting backend, where the rules are), and `apphosting.yaml` (build-time config) and picks up where you left off.
- **No special phrasing.** "*Publish my app*", "*Deploy only my Firestore rules*", or "*Run the build and then deploy to production*" all work.
- **New environment variables go in two different places.** A `NEXT_PUBLIC_*` value the static build needs goes in `apphosting.yaml` as a plain `value:`. A real secret stays in `.env.local` locally and

goes to App Hosting via `firebase apphosting:secrets:set` with a `secret:` reference in `apphosting.yaml`. Forgetting the first is the build failure described above; putting the second in `value:` leaks it into a committed file.

- **Commit and push after each session you're happy with**, so the GitHub copy stays current. `firebase deploy` doesn't touch git and git doesn't trigger a deploy unless you've connected the backend to the repo (Section 10, Path A) — they're independent until you wire them together.

Continue at Section 5.

---

4. Path 2 — Start a New Project in Antigravity

Create the Firebase project

Do this in the Firebase console first: **Add project**, name it, decide on Analytics. If you plan to use App Hosting (the default for Next.js), upgrade to the **Blaze** plan now — App Hosting requires it.

Scaffold the app

Pick a framework. These are the three the Firebase tooling supports best:

```
# Next.js (App Router, TypeScript) — best fit for App Hosting
npx create-next-app@latest my-app --typescript --app --eslint
# Vite + React — best fit for static Hosting
npm create vite@latest my-app -- --template react-ts
# Angular
npx @angular/cli@latest new my-app
```

Each scaffolder runs `npm install` for you as its last step, so `node_modules/` is already populated. If you cloned an existing repo instead of scaffolding, run `npm install` first (details under "Install the project's dependencies" in Section 3). Then open the folder in Antigravity.

Initialize Firebase — Option A, agent-driven

With the Firebase Bundle and MCP server installed (Section 2), prompt:

> *"Initialize Firebase in this project using my `<project-id>` project. Set up Firestore, Authentication with Google sign-in, and the local emulators."*

The agent will register a web app in your Firebase project (or reuse one), fetch its config, write the SDK initialization code, create `firebase.json` / `.firebaserc` / `firestore.rules`, and present a plan before executing. Review it, then approve.

Initialize Firebase — Option B, manual

```
firebase init
```

Select the features you need. For a typical app: **Firestore**, **Emulators**, and either **App Hosting** or **Hosting**. Accept the defaults for rules and index file names.

Then register a web app and grab its config from **Project settings → Your apps → Add app (Web)** in the console, and initialize the SDK:

```
npm install firebase
```

```
// src/lib/firebase.ts (or .js)
import { initializeApp } from 'firebase/app';
import { getFirestore, connectFirestoreEmulator } from 'firebase/firestore';
import { getAuth, connectAuthEmulator } from 'firebase/auth';

const firebaseConfig = {
  apiKey: process.env.NEXT_PUBLIC_FIREBASE_API_KEY,
  authDomain: process.env.NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN,
  projectId: process.env.NEXT_PUBLIC_FIREBASE_PROJECT_ID,
  storageBucket: process.env.NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: process.env.NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID,
  appId: process.env.NEXT_PUBLIC_FIREBASE_APP_ID
};

const app = initializeApp(firebaseConfig);
export const db = getFirestore(app);
export const auth = getAuth(app);

// Talk to the local emulators in development
if (process.env.NODE_ENV === 'development') {
  connectFirestoreEmulator(db, '127.0.0.1', 8080);
  connectAuthEmulator(auth, 'http://127.0.0.1:9099');
}
```

(For Vite, use `import.meta.env.VITE_*` instead of `process.env.NEXT_PUBLIC_*`.) Put the values in `.env.local` and keep that file out of git. The web API key is not a secret — it's shipped to every browser — but keeping config out of source is still good hygiene.

Build features with the agent

From here, development is prompt-driven. Some starting points:

> *"Create a `users` collection with a signup form that stores email and username in Firestore. Validate the inputs client-side and write matching security rules."*

> *"Add Google sign-in with a header showing the signed-in user's avatar and a sign-out button."*

The agent writes code, rules, and tests together when you ask it to. Always ask for the rules alongside the feature — it's the step most often skipped.

Continue at Section 5.

---

5. Working With the Agent: Prompts That Do Real Work

These prompts apply to both paths. Phrase them your own way; the agent understands intent, not incantations.

Audit security rules

> "*Scan `firestore.rules`. Flag any rule that allows unauthenticated writes, any collection with no rule at all, and any rule that lets a user modify a document they don't own. Propose fixes and show me a diff.*"

The agent locates the rules file from `firebase.json`, walks each `match` block, and prints current-vs-proposed rules in the chat. The full rules blueprint in Section 7 is a good baseline to hand it.

One finding to expect on Studio-generated projects: a legacy collection with no rule at all (Studio prototypes often leave a top-level collection behind). Firestore denies any path no rule matches, so this is secure as-is, but the agent will suggest an explicit catch-all so the intent is visible to the next person reading the file:

```
match /databases/{database}/documents {
  // Explicit default-deny for anything not matched below.
  match /{document=**} {
    allow read, write: if false;
  }
  // ...specific match blocks follow
}
```

Worth accepting. Just understand what it does and doesn't do: Firestore grants access if *any* matching rule allows it, so this block never overrides a specific `allow` — it only documents the default. Your real protection is still the per-collection rules. Ask the agent to "*apply the rule*" and it edits the file; it isn't live until you deploy (see below).

Verify rules against the emulator

> "*Start the Firestore emulator and write a rules unit test that proves an anonymous user cannot read `/users/{id}`.*"

Or do it yourself with `firebase emulators:start --only firestore` and use the Emulator UI at `http://localhost:4000`, which has a rules playground. Section 9 has a full test harness.

Pre-deploy check

> "*Prepare this app for deployment. Run the production build, check that `firestore.rules` is registered in `firebase.json`, confirm no secrets are in source, and list any environment variables I still need to set.*"

Reading the result: a `next build` that logs warnings about large images or PDFs not being precached by the service worker is still a passing build. The environment-variable list it produces is the set you'll need in `apphosting.yaml` (public `NEXT_PUBLIC_*` config) or Secret Manager (`GEMINI_API_KEY` and any other server-side key) — and if a key has been sitting in `.env.local` on a Studio workspace for months, rotate it before it goes to production. The "no secrets in source" check only inspects files git tracks, so it proves nothing until you've run `git init`.

Deploy

> "*Publish my app*" — full deploy (build + hosting + rules).

> "*Deploy only my Firestore rules*" — or run `firebase deploy --only firestore:rules` yourself. This replaces the live rules immediately; there is no staging step for rules.

When the agent gets it wrong

Tell it what's wrong and ask it to revise the plan rather than fixing files by hand and then continuing. It loses track of manual edits it didn't make.

---

6. Choose Your Deployment Path

This is the decision that shapes everything downstream, so make it deliberately.

	App Hosting	Firebase Hosting (static)
What it is	Server-rendered hosting on Cloud Run with built-in CDN and GitHub rollouts	CDN for pre-built static files
Fits	Next.js / Angular apps with server components, API routes, server actions, Genkit flows, or middleware	Pure client-side SPAs: Vite/React, Angular without SSR, Next.js with `output: 'export'`
Firebase Studio default	✅ Apps built with the App Prototyping agent deploy here	❌
Billing	Requires the Blaze (pay-as-you-go) plan	Free tier available
CI/CD	Built in — push to the live branch triggers a rollout	You wire it up yourself (see Section 10)

Rule of thumb: if your app came from Studio's App Prototyping agent, it's a Next.js app already deployed to App Hosting. Stay there unless you have a specific reason to go static. Forcing `output: 'export'` onto an app that uses any server-side feature will break it at build time.

Path A — App Hosting

App Hosting is configured with an `apphosting.yaml` in your project root, not `firebase.json`. If the export didn't leave one behind, scaffold it:

```
firebase init apphosting
```

A minimal `apphosting.yaml`:

```
runConfig:
  minInstances: 0
  maxInstances: 4

env:
  - variable: NODE_ENV
    value: production
```

Secrets should reference Cloud Secret Manager rather than being pasted in:

```
env:
  - variable: GEMINI_API_KEY
    secret: gemini-api-key
```

Create the secret with `firebase apphosting:secrets:set gemini-api-key`.

Your public `NEXT_PUBLIC_FIREBASE_*` config also belongs in this file, as plain `value:` entries. The cloud builder doesn't see your `.env.local`, and Next.js needs those values while prerendering, so a build that passes locally will fail in the cloud without them (the full fix is under "Republish" in Section 3).

First deploy from the CLI. `firebase deploy` walks you through a few prompts when it can't find a backend: pick a **region** (match your Firestore region — usually `us-central1` — unless you have a reason not to), accept the default **Node.js runtime**, and confirm the backend name. If a Studio-created backend already exists, don't create a new one — `firebase apphosting:backends:list` shows what's there, and the agent needs to be told about it explicitly.

Then either let the agent handle it (`Publish my app`) or connect the backend to GitHub yourself — see Section 10, Path A.

Path B — Firebase Hosting (static)

Your `firebase.json` maps the build output directory to Hosting. Pick the block that matches your framework.

Vite / React — build output is `dist`:

```
{
  "hosting": {
    "public": "dist",
    "ignore": ["firebase.json", "**/.*", "**/node_modules/**"],
    "rewrites": [
      { "source": "**", "destination": "/index.html" }
    ]
  }
}
```

Next.js static export — set `output: 'export'` in `next.config.js`; build output is `out`. Don't add a catch-all rewrite: Hosting serves `out/404.html` automatically for missing paths, and rewriting `**` to it would send every route to the 404 page.

```
{
  "hosting": {
    "public": "out",
    "ignore": ["firebase.json", "**/.*", "**/node_modules/**"],
    "cleanUrls": true,
    "trailingSlash": false
  }
}
```

Angular — output is `dist/<project-name>/browser`:

```
{
  "hosting": {
    "public": "dist/your-project-name/browser",
    "ignore": ["firebase.json", "**/.*", "**/node_modules/**"],
    "rewrites": [
      { "source": "**", "destination": "/index.html" }
    ]
  }
}
```

---

7. Production Firestore Security Rules

Create `firestore.rules` in the project root. This schema isolates per-user documents, validates input shape at the database boundary, and prevents ownership transfer on posts.

```
rules_version = '2';

service cloud.firestore {
  match /databases/{database}/documents {

    // ---- Helpers ----
    function isAuthenticated() {
      return request.auth != null;
    }

    function isOwner(userId) {
      return isAuthenticated() && request.auth.uid == userId;
    }

    function isValidEmail(email) {
      return email.matches("^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\\.[a-zA-Z]{2,}$");
    }

    function isValidUsername(username) {
      return username.size() >= 3 && username.size() <= 20;
    }

    // ---- Users ----
    match /users/{userId} {
      // NOTE: any signed-in user can read any profile, including its email.
      // Tighten to isOwner(userId) if profiles should be private.
      allow read: if isAuthenticated();

      allow create: if isOwner(userId)
                    && isValidEmail(request.resource.data.email)
                    && isValidUsername(request.resource.data.username);

      // Email is frozen after creation
      allow update: if isOwner(userId)
                    && request.resource.data.email == resource.data.email;

      // No client-side deletes; handle account deletion server-side
      allow delete: if false;
    }

    // ---- Posts ----
    match /posts/{postId} {
      allow read: if true;

      allow create: if isAuthenticated()
                    && request.resource.data.authorId == request.auth.uid
                    && request.resource.data.title is string
                    && request.resource.data.title.size() > 0;

      allow update: if isAuthenticated()
                    && resource.data.authorId == request.auth.uid
                    && request.resource.data.authorId == resource.data.authorId;

      allow delete: if isAuthenticated() && resource.data.authorId == request.auth.uid;
```

```
    }
  }
}
```

Register the rules file in `firebase.json` so both the emulator and `firebase deploy` pick it up. This block coexists with any `hosting` or `emulators` blocks you already have:

```
{
  "firestore": {
    "rules": "firestore.rules",
    "indexes": "firestore.indexes.json"
  }
}
```

If you don't have a `firestore.indexes.json` yet, `firebase init firestore` creates both files.

---

8. Local Emulator Seeding

The Local Emulator Suite lets you develop and test against Firestore without touching production data or incurring reads.

Configure the emulators

Add an `emulators` block to `firebase.json` (merge with existing keys; don't overwrite the file):

```
{
  "emulators": {
    "firestore": { "port": 8080 },
    "auth": { "port": 9099 },
    "ui": { "enabled": true, "port": 4000 }
  }
}
```

Or run `firebase init emulators` and pick Firestore, Authentication, and the UI.

Write the seed script

Install the Admin SDK as a dev dependency — it's only used for seeding, never in the browser bundle:

```
npm install --save-dev firebase-admin
```

Create `scripts/seed.js`:

```
const { initializeApp } = require('firebase-admin/app');
const { getFirestore } = require('firebase-admin/firestore');

// Point the Admin SDK at the local emulator instead of production
process.env.FIRESTORE_EMULATOR_HOST = '127.0.0.1:8080';

initializeApp({ projectId: 'demo-antigravity-migration' });
const db = getFirestore();

async function runSeeding() {
  console.log('Seeding local Firestore...');

  const sampleUsers = [
    {
      id: 'user_alice',
      data: {
        username: 'alice_dev',
        email: 'alice@example.com',
        createdAt: new Date().toISOString()
      }
    },
    {
      id: 'user_bob',
      data: {
        username: 'bob_tester',
        email: 'bob@example.com',
        createdAt: new Date().toISOString()
      }
    }
  ];

  for (const user of sampleUsers) {
    await db.collection('users').doc(user.id).set(user.data);
  }

  console.log('Done.');
}

runSeeding().catch((err) => {
  console.error(err);
  process.exit(1);
});
```

> **Why `demo-` as the project ID prefix?** The Firebase CLI treats any project ID starting with `demo-` as emulator-only and refuses to make production calls with it. That's a safety net worth keeping.

Capture a reusable snapshot

1. Start the emulators:

```
firebase emulators:start --project demo-antigravity-migration
```

2. In a second terminal, run the seed script:

```
node scripts/seed.js
```

3. In a third terminal, export the emulator state to disk:

```
firebase emulators:export ./firebase-seed-data --project demo-antigravity-migration
```

4. From now on, start with the snapshot loaded — and persist any changes you make during the session on exit:

```
firebase emulators:start \
  --project demo-antigravity-migration \
  --import=./firebase-seed-data \
  --export-on-exit
```

Add `firebase-seed-data/` to `.gitignore` unless you want the fixture committed.

---

9. Firestore Rules Unit Tests

The `@firebase/rules-unit-testing` library runs your real `firestore.rules` against the emulator, so you can assert what's allowed and denied before anything ships.

Install

```
npm install --save-dev jest @firebase/rules-unit-testing
```

Add a script to `package.json`. `emulators:exec` boots the Firestore emulator, runs the command, then shuts it down:

```
{
  "scripts": {
    "test:rules": "firebase emulators:exec --only firestore --project demo-antigravity-migration
'jest --runInBand'"
  }
}
```

Write the tests

Create `firestore.test.js` in the project root:

```
const {
  initializeTestEnvironment,
  assertFails,
  assertSucceeds
} = require('@firebase/rules-unit-testing');
const fs = require('fs');

let testEnv;

beforeAll(async () => {
  testEnv = await initializeTestEnvironment({
    projectId: 'demo-antigravity-migration',
    firestore: {
      rules: fs.readFileSync('firestore.rules', 'utf8')
      // host/port are picked up from FIRESTORE_EMULATOR_HOST,
      // which emulators:exec sets automatically
    }
  });
});

beforeEach(async () => {
  await testEnv.clearFirestore();
});

afterAll(async () => {
  await testEnv.cleanup();
});

describe('users collection', () => {
  test('denies anonymous reads', async () => {
    const db = testEnv.unauthenticatedContext().firestore();
    await assertFails(db.collection('users').doc('user_alice').get());
  });

  test('allows authenticated reads', async () => {
    const db = testEnv.authenticatedContext('user_bob').firestore();
    await assertSucceeds(db.collection('users').doc('user_alice').get());
  });

  test('denies writing to another user\'s document', async () => {
    const db = testEnv.authenticatedContext('user_attacker').firestore();
    await assertFails(
      db.collection('users').doc('user_alice').set({
        username: 'hacked_alice',
        email: 'attacker@evil.com'
      })
    );
  });

  test('denies creation with a malformed email', async () => {
    const db = testEnv.authenticatedContext('user_alice').firestore();
    await assertFails(
      db.collection('users').doc('user_alice').set({
        username: 'alice_dev',
```

```
        email: 'not-an-email'
      })
    );
  });

  test('allows creation when the schema is valid', async () => {
    const db = testEnv.authenticatedContext('user_alice').firestore();
    await assertSucceeds(
      db.collection('users').doc('user_alice').set({
        username: 'alice_dev',
        email: 'alice@example.com'
      })
    );
  });

  test('denies changing email after creation', async () => {
    const db = testEnv.authenticatedContext('user_alice').firestore();
    const doc = db.collection('users').doc('user_alice');
    await assertSucceeds(doc.set({ username: 'alice_dev', email: 'alice@example.com' }));
    await assertFails(doc.update({ email: 'new@example.com' }));
  });
});
```

Run:

```
npm run test:rules
```

---

10. Continuous Deployment

Path A — App Hosting (built in)

App Hosting has GitHub integration built into the backend itself. There's nothing to write in `.github/workflows/`.

1. Push your project to a GitHub repository.
2. In the Firebase console go to **Hosting & Serverless** → **App Hosting** → **Get started** (or run `firebase apphosting:backends:create`).
3. **Connect to GitHub**, authorize the Firebase GitHub app, and pick the repo.
4. Set the **root directory** (where `package.json` lives, usually `/`) and the **live branch** (usually `main`).
5. Leave **automatic rollouts** enabled.

From then on, every push to the live branch triggers a Cloud Build and a rollout. You can disable auto-rollouts in the backend's **Deployment settings** tab and trigger manually instead with:

```
firebase apphosting:rollouts:create BACKEND_ID --project PROJECT_ID
```

Firestore rules aren't part of an App Hosting rollout. Deploy them separately when they change:

```
firebase deploy --only firestore:rules
```

If you want rules tests and rules deployment gated in CI too, add the small workflow at the end of Path B — just drop the hosting step.

Path B — Firebase Hosting via GitHub Actions

Service account

The simplest route is to let the CLI set everything up:

```
firebase init hosting:github
```

It creates a scoped service account, stores the key as a repository secret named `FIREBASE_SERVICE_ACCOUNT_<PROJECT_ID>`, and writes a starter workflow.

If you'd rather do it by hand, create a service account in the Google Cloud Console with **only** these roles (this is the set the deploy action's own documentation specifies):

Role	ID	Why
Firebase Hosting Admin	`roles/firebasehosting.admin`	Deploy to Hosting channels
Firebase Authentication Admin	`roles/firebaseauth.admin`	Add preview URLs to Auth's authorized domains
Cloud Run Viewer	`roles/run.viewer`	Only if Hosting rewrites to Cloud Run / Functions
API Keys Viewer	`roles/serviceusage.apiKeysViewer`	Required by the CLI
Firebase Rules Admin	`roles/firebaserules.admin`	Only if the workflow also deploys `firestore.rules`

Download a JSON key and add it as a GitHub secret under **Settings → Secrets and variables → Actions**. Avoid granting broad roles like Firebase Admin or Owner to a key that lives in GitHub.

Workflow

Create `.github/workflows/deploy.yml`:

```yaml
name: Deploy

on:
  push:
    branches: [main]

jobs:
  test-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm

      - run: npm ci

      # Runs the Firestore emulator; ubuntu-latest ships with a JDK
      - name: Test security rules
        run: npm run test:rules

      - name: Build
        run: npm run build

      - name: Deploy Firestore rules
        run: |
          echo '${{ secrets.FIREBASE_SERVICE_ACCOUNT }}' > "$RUNNER_TEMP/sa.json"
          export GOOGLE_APPLICATION_CREDENTIALS="$RUNNER_TEMP/sa.json"
          npx firebase-tools deploy --only firestore:rules --project "${{
secrets.FIREBASE_PROJECT_ID }}"

      - name: Deploy Hosting
        uses: FirebaseExtended/action-hosting-deploy@v0
        with:
          repoToken: ${{ secrets.GITHUB_TOKEN }}
          firebaseServiceAccount: ${{ secrets.FIREBASE_SERVICE_ACCOUNT }}
          projectId: ${{ secrets.FIREBASE_PROJECT_ID }}
          channelId: live
```

Two things to know about this file:

- `action-hosting-deploy` deploys **Hosting only**. That's why there's a separate rules step — without it, the rules you tested never reach production. That step needs the Firebase Rules Admin role from the table above.
- The rules step authenticates the CLI by writing the service account JSON to a temp file and pointing `GOOGLE_APPLICATION_CREDENTIALS` at it. `action-hosting-deploy` does the same thing internally; it just accepts the JSON directly.

---

11. Next Steps

- If you deployed via `firebase deploy` from your machine, push the repo to GitHub and connect it to your App Hosting backend so you stop depending on one laptop for releases.
- Put the app on your own domain instead of the `hosted.app` URL. The companion guide, Connecting a Custom Domain to Firebase App Hosting, walks through custom domains and subdomains with Cloudflare DNS.
- Let users store files. Private Cloud Storage for a Next.js Site on Firebase App Hosting sets up a private bucket with signed upload and download links, including the size limits that break large uploads on App Hosting.
- Add rules tests for your app's real collections and fields — timestamps, enums, string length limits, and any cross-document references.
- Wire the Auth emulator into the seed script so you can test third-party sign-in flows (Google, Apple) offline.
- If you're on App Hosting, look at `apphosting.staging.yaml` and a second backend for a staging environment before you scale up.
- Move the email and username validators into a shared `firestore.rules` function file only if your rules grow large enough to warrant it — premature abstraction in rules makes them harder to audit.

Sources

- Antigravity docs: Firebase Studio Migration
- Antigravity docs: Build with Google
- Codelab: How to Migrate from Firebase Studio to Antigravity
- Firebase: Studio sunset and project migration
- Firebase MCP server
- Firebase App Hosting: Get started
- action-hosting-deploy: service account roles

Latest version: github.com/ImmuneMoon/Building-Firebase-Apps-in-Google-Antigravity-IDE

If this helped, you can support Fulllion Creative Works through PayPal or Ko-fi.

© 2026 Fulllion Creative Works. All rights reserved.